

IOT Unit 4

Data and Analytics for IoT

In one of the famous episodes of the classic American science fiction TV series *Star Trek*, a harmless furry alien creature known as a “tribble” is brought aboard the starship *Enterprise*. At first, the cute little tribble is treated like a pet, but then its unusual property shows up: It is able to multiply itself at an alarming rate, to the point that the ship soon becomes so filled with tribbles that they consume all supplies on board and begin interfering with the ship’s systems.

The problems of data generated by IoT networks might well resemble “The Trouble with Tribbles.” At first, IoT data is just a curiosity, and it’s even useful if handled correctly. However, given time, as more and more devices are added to IoT networks, the data generated by these systems becomes overwhelming. Not only does this data begin to consume precious network bandwidth but server resources are increasingly taxed in their attempt to process, sort, and analyze the data.

Traditional data management systems are simply unprepared for the demands of what has come to be known as “big data.” As discussed throughout this book, the real value of IoT is not just in connecting things but rather in the data produced by those things, the new services you can enable via those connected things, and the business insights that the data can reveal. However, to be useful, the data needs to be handled in a way that is organized and controlled. Thus, a new approach to data analytics is needed for the Internet of Things.

This chapter provides an overview of the field of data analytics from an IoT perspective, including the following sections:

- **An Introduction to Data Analytics for IoT:** This section introduces the subject of analytics for IoT and discusses the differences between structured and unstructured data. It also discusses how analytics relates to IoT data.
- **Machine Learning:** Once you have the data, what do you do with it, and how can you gain business insights from it? This section delves into the major types of machine learning that are used to gain business insights from IoT data.
- **Big Data Analytics Tools and Technology:** *Big data* is one of the most commonly used terms in the world of IoT. This section examines some of the most common technologies used in big data today, including Hadoop, NoSQL, MapReduce, and MPP.
- **Edge Streaming Analytics:** IoT requires that data be processed and analyzed as close to the endpoint as possible, in real-time. This section explores how streaming analytics can be used for
 - such processing and analysis.
- **Network Analytics:** The final section of this chapter investigates the concept of network flow analytics using Flexible NetFlow in IoT systems. NetFlow can help you better understand the function of the overall system and heighten security in an IoT network.

An Introduction to Data Analytics for IoT

In the world of IoT, the creation of massive amounts of data from sensors is common and one of the biggest challenges—not only from a transport perspective but also from a data management standpoint. A great example of the deluge of data that can be generated by IoT is found in the commercial aviation industry and the sensors that are deployed throughout an aircraft.

Modern jet engines are fitted with thousands of sensors that generate a whopping 10GB of data per second. For example, modern jet engines, similar to the one shown in Figure 7-1, may be equipped with around 5000 sensors. Therefore, a twin engine commercial aircraft with these engines operating on average 8 hours a day will generate over 500 TB of data daily, and this is just the data from the engines! Aircraft today have thousands of other sensors connected to the airframe and other systems. In fact, a single wing of a modern jumbo jet is equipped with 10,000 sensors.



Figure 7-1 *Commercial Jet Engine*

The potential for a petabyte (PB) of data per day per commercial airplane is not farfetched—and this is just for *one* airplane. Across the world, there are approximately 100,000 commercial flights per day. The amount of IoT data coming just from the commercial airline business is overwhelming.

This example is but one of many that highlight the big data problem that is being exacerbated by IoT. Analyzing this amount of data in the most efficient manner possible falls under the umbrella of data analytics. Data analytics must be able to offer actionable insights and

knowledge from data, no matter the amount or style, in a timely manner, or the full benefits of IoT cannot be realized.

Note

Another example regarding the amount of data being generated by IoT, and thus the need for data analytics, is the utility industry. Even moderately sized smart meter networks can provide over 1 billion data points each day.

Before diving deeper into data analytics, it is important to define a few key concepts related to data. For one thing, not all data is the same; it can be categorized and thus analyzed in different ways. Depending on how data is categorized, various data analytics tools and processing methods can be applied. Two important categorizations from an IoT perspective are whether the data is structured or unstructured and whether it is in motion or at rest.

Structured Versus Unstructured Data

Structured data and unstructured data are important classifications as they typically require different toolsets from a data analytics perspective. [Figure 7-2](#) provides a high-level comparison of structured data and unstructured data.

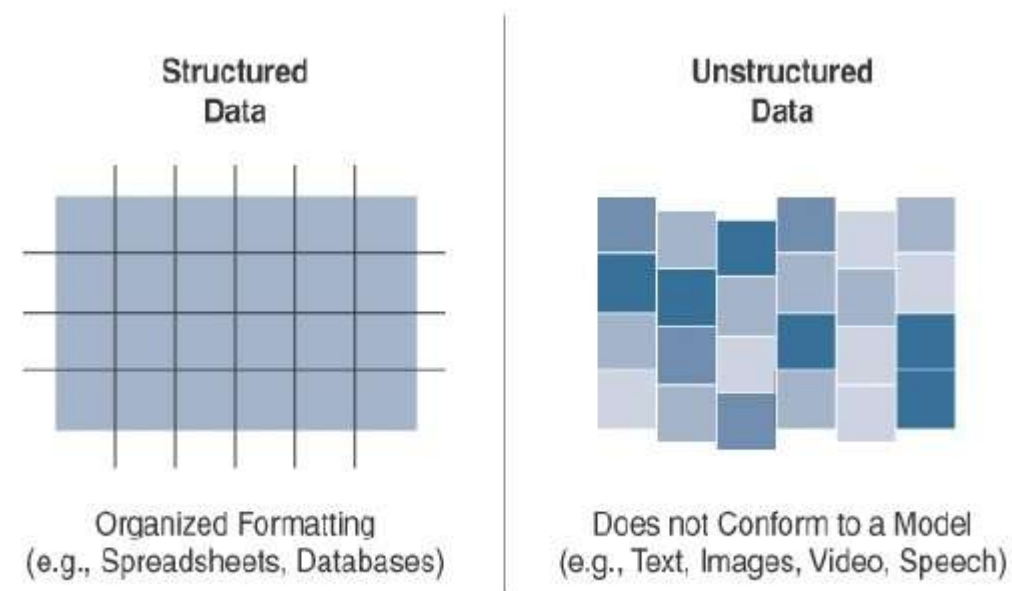


Figure 7-2 *Comparison Between Structured and Unstructured Data*

Structured data means that the data follows a model or schema that defines how the data is represented or organized, meaning it fits well with a traditional relational database management system (RDBMS). In many cases you will find structured data in a simple tabular form—for example, a spreadsheet where data occupies a specific cell and can be explicitly defined and referenced.

Structured data can be found in most computing systems and includes everything from banking transaction and invoices to computer log files and router configurations. IoT sensor data often uses structured values, such as temperature, pressure, humidity, and so on, which are all sent

in a known format. Structured data is easily formatted, stored, queried, and processed; for these reasons, it has been the core type of data used for making business decisions.

Because of the highly organizational format of structured data, a wide array of data analytics tools are readily available for processing this type of data. From custom scripts to commercial software like Microsoft Excel and Tableau, most people are familiar and comfortable with working with structured data.

Unstructured data lacks a logical schema for understanding and decoding the data through traditional programming means. Examples of this data type include text, speech, images, and video. As a general rule, any data that does not fit neatly into a predefined data model is classified as unstructured data.

According to some estimates, around 80% of a business's data is unstructured. Because of this fact, data analytics methods that can be applied to unstructured data, such as cognitive computing and machine learning, are deservedly garnering a lot of attention. With machine learning applications, such as natural language processing (NLP), you can decode speech. With image/facial recognition applications, you can extract critical information from still images and video. The handling of unstructured IoT data employing machine learning techniques is covered in more depth later in this chapter.

Note

A third data classification, semi-structured data, is sometimes included along with structured and unstructured data. As you can probably guess, semi-structured data is a hybrid of structured and unstructured data and shares characteristics of both. While not relational, semi-structured data contains a certain schema and consistency. Email is a good example of semi-structured data as the fields are well defined but the content contained in the body field and attachments is unstructured. Other examples include JavaScript Object Notation (JSON) and Extensible Markup Language (XML), which are common data interchange formats used on the web and in some IoT data exchanges.

Smart objects in IoT networks generate both structured and unstructured data. Structured data is more easily managed and processed due to its well-defined organization. On the other hand, unstructured data can be harder to deal with and typically requires very different analytics tools for processing the data. Being familiar with both of these data classifications is important because knowing which data classification you are working with makes integrating with the appropriate data analytics solution much easier.

Data in Motion Versus Data at Rest

As in most networks, data in IoT networks is either in transit ("data in motion") or being held or stored ("data at rest"). Examples of data in motion include traditional client/server exchanges, such as web browsing and file transfers, and email. Data saved to a hard drive, storage array, or USB drive is data at rest.

From an IoT perspective, the data from smart objects is considered data in motion as it passes through the network en route to its final destination. This is often processed at the edge, using fog computing. When data is processed at the edge, it may be filtered and deleted or forwarded on for further processing and possible storage at a fog node or in the data center. Data does not come to rest at the edge.

When data arrives at the data center, it is possible to process it in real-time, just like at the edge, while it is still in motion. Tools with this sort of capability, such as Spark, Storm, and Flink, are relatively nascent compared to the tools for analyzing stored data. Later sections of this chapter provide more information on these real-time streaming analysis tools that are part of the Hadoop ecosystem.

Data at rest in IoT networks can be typically found in IoT brokers or in some sort of storage array at the data center. Myriad tools, especially tools for structured data in relational databases, are available from a data analytics perspective. The best known of these tools is Hadoop. Hadoop not only helps with data processing but also data storage.

IoT Data Analytics Overview

The true importance of IoT data from smart objects is realized only when the analysis of the data leads to actionable business intelligence and insights. Data analysis is typically broken down by the types of results that are produced. As shown in [Figure 7-3](#), there are four types of data analysis results:

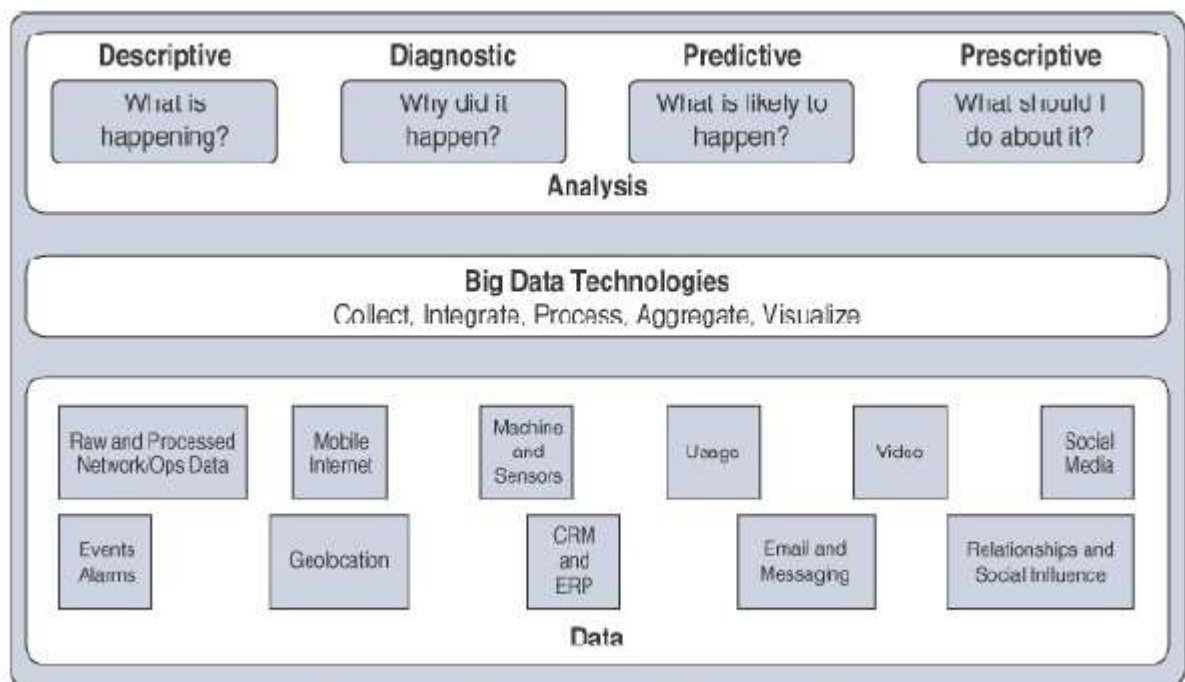


Figure 7-3 *Types of Data Analysis Results*

- **Descriptive:** Descriptive data analysis tells you what is happening, either now or in the past. For example, a thermometer in a truck engine reports temperature values every second. From a descriptive analysis perspective, you can pull this data at any moment to gain insight into the current operating condition of the truck engine. If the temperature value is too high, then there may be a cooling problem or the engine may be experiencing too much load.
- **Diagnostic:** When you are interested in the “why,” diagnostic data analysis can provide the answer. Continuing with the example of the temperature sensor in the truck engine, you might wonder why the truck engine failed. Diagnostic analysis might show that the temperature of the engine was too high, and the engine overheated. Applying diagnostic analysis across the data generated by a wide range of smart objects can provide a clear picture of why a problem or an event occurred.
- **Predictive:** Predictive analysis aims to foretell problems or issues before they occur. For example, with historical values of temperatures for the truck engine, predictive analysis could provide an estimate on the remaining life of certain components in the engine. These components could then be proactively replaced before failure occurs. Or perhaps if temperature values of the truck engine start to rise slowly over time, this could indicate the need for an oil change or some other sort of engine cooling maintenance.
- **Prescriptive:** Prescriptive analysis goes a step beyond predictive and recommends solutions for upcoming problems. A prescriptive analysis of the temperature data from a truck engine might calculate various alternatives to cost-effectively maintain our truck. These calculations could range from the cost necessary for more frequent oil changes and cooling maintenance to installing new cooling equipment on the engine or upgrading to a lease on a model with a more powerful engine. Prescriptive analysis looks at a variety of factors and makes the appropriate recommendation.

Both predictive and prescriptive analyses are more resource intensive and increase complexity, but the value they provide is much greater than the value from descriptive and diagnostic analysis. [Figure 7-4](#) illustrates the four data analysis types and how they rank as complexity and value increase. You can see that descriptive analysis is the least complex and at the same time offers the least value. On the other end, prescriptive analysis provides the most value but is the most complex to implement. Most data analysis in the IoT space relies on descriptive and diagnostic analysis, but a shift toward predictive and prescriptive analysis is understandably occurring for most businesses and organizations.

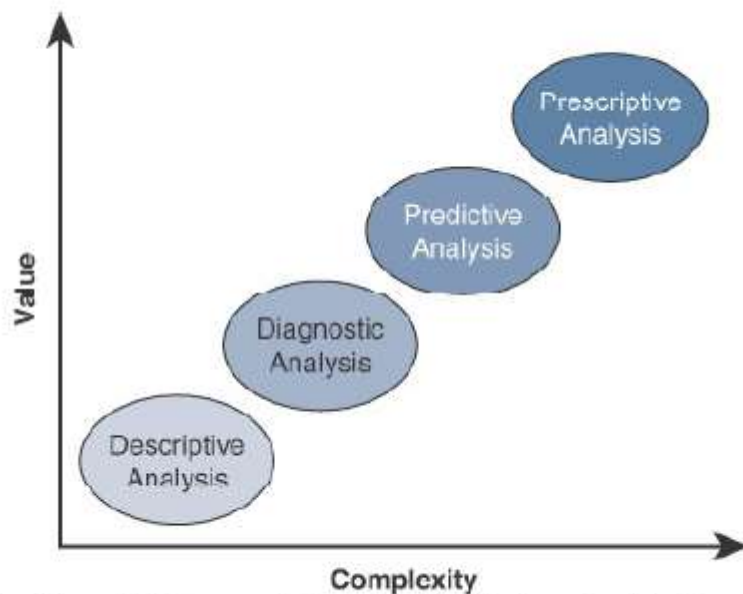


Figure 7-4 *Application of Value and Complexity Factors to the Types of Data Analysis*

IoT Data Analytics Challenges

As IoT has grown and evolved, it has become clear that traditional data analytics solutions were not always adequate. For example, traditional data analytics typically employs a standard RDBMS and corresponding tools, but the world of IoT is much more demanding. While relational databases are still used for certain data types and applications, they often struggle with the nature of IoT data. IoT data places two specific challenges on a relational database:

- **Scaling problems:** Due to the large number of smart objects in most IoT networks that continually send data, relational databases can grow incredibly large very quickly. This can result in performance issues that can be costly to resolve, often requiring more hardware and architecture changes.
- **Volatility of data:** With relational databases, it is critical that the schema be designed correctly from the beginning. Changing it later can slow or stop the database from operating. Due to the lack of flexibility, revisions to the schema must be kept at a minimum. IoT data, however, is volatile in the sense that the data model is likely to change and evolve over time. A dynamic schema is often required so that data model changes can be made daily or even hourly.

To deal with challenges like scaling and data volatility, a different type of database, known as NoSQL, is being used. Structured Query Language (SQL) is the computer language used to communicate with an RDBMS. As the name implies, a NoSQL database is a database that does not use SQL. It is not set up in the traditional tabular form of a relational database. NoSQL

databases do not enforce a strict schema, and they support a complex, evolving data model. These databases are also inherently much more scalable.

In addition to the relational database challenges that IoT imposes, with its high volume of smart object data that frequently changes, IoT also brings challenges with the live streaming nature of its data and with managing data at the network level. Streaming data, which is generated as smart objects transmit data, is challenging because it is usually of a very high volume, and it is valuable only if it is possible to analyze and respond to it in real-time. Real-time analysis of streaming data allows you to detect patterns or anomalies that could indicate a problem or a situation that needs some kind of immediate response. To have a chance of affecting the outcome of this problem, you naturally must be able to filter and analyze the data while it is occurring, as close to the edge as possible.

The market for analyzing streaming data in real-time is growing fast. Major cloud analytics providers, such as Google, Microsoft, and IBM, have streaming analytics offerings, and various other applications can be used in house.

Another challenge that IoT brings to analytics is in the area of network data, which is referred to as *network analytics*. With the large numbers of smart objects in IoT networks that are communicating and streaming data, it can be challenging to ensure that these data flows are effectively managed, monitored, and secure. Network analytics tools such as Flexible NetFlow and IPFIX provide the capability to detect irregular patterns or other problems in the flow of IoT data through a network.

Machine Learning

One of the core subjects in IoT is how to make sense of the data that is generated. Because much of this data can appear incomprehensible to the naked eye, specialized tools and algorithms are needed to find the data relationships that will lead to new business insights. This brings us to the subject of machine learning (ML).

Machine learning, deep learning, neural networks, and convolutional networks are words you have probably heard in relation to big data and IoT. ML is indeed central to IoT. Data collected by smart objects needs to be analyzed, and intelligent actions need to be taken based on these analyses. Performing this kind of operation manually is almost impossible (or very, very slow and inefficient). Machines are needed to process information fast and react instantly when thresholds are met. For example, every time a new advance is made in the field of self-driving vehicles, abnormal pattern recognition in a crowd, or any other automated intelligent and machine-assisted decision system, ML is named as the tool that made the advance possible. But ML is not new. It was invented in the middle of the twentieth century and actually fell out of fashion in the 1980s. So what has happened in ML that makes it the new tool of choice for IoT and data analytics?

Machine Learning Overview

Machine learning is, in fact, part of a larger set of technologies commonly grouped under the term *artificial intelligence* (AI). This term used to make science fiction amateurs dream of biped robots and conscious machines, or of a *Matrix*-like world where machines would enslave humankind. In fact, AI includes any technology that allows a computing system to mimic human intelligence using any technique, from very advanced logic to basic “if-then-else” decision loops. Any computer that uses rules to make decisions belongs to this realm. A simple example is an app that can help you find your parked car. A GPS reading of your position at regular intervals calculates your speed. A basic threshold system determines whether you are driving (for example, “if speed > 20 mph or 30 kmh, then start calculating speed”). When you park and disconnect from the car Bluetooth system, the app simply records the location when the disconnection happens. This is where your car is parked. Beyond the appearance of artificial intelligence (the computer knows that you are parked and where this happened), the ruleset is very simple.

In more complex cases, static rules cannot be simply inserted into the program because they require parameters that can change or that are imperfectly understood. A typical example is a dictation program that runs on a computer. The program is configured to recognize the audio pattern of each word in a dictionary, but it does not know your voice’s specifics—your accent, tone, speed, and so on. You need to record a set of predetermined sentences to help the tool match well-known words to the sounds you make when you say the words. This process is called machine learning. ML is concerned with any process where the computer needs to receive a set of data that is processed to help perform a task with more efficiency. ML is a vast field but can be simply divided in two main categories: supervised and unsupervised learning.

Supervised Learning

In supervised learning, the machine is trained with input for which there is a known correct answer. For example, suppose that you are training a system to recognize when there is a human in a mine tunnel. A sensor equipped with a basic camera can capture shapes and return them to a computing system that is responsible for determining whether the shape is a human or something else (such as a vehicle, a pile of ore, a rock, a piece of wood, and so on.). With supervised learning techniques, hundreds or thousands of images are fed into the machine, and each image is labeled (human or nonhuman in this case). This is called the *training set*. An algorithm is used to determine common parameters and common differences between the images. The comparison is usually done at the scale of the entire image, or pixel by pixel. Images are resized to have the same characteristics (resolution, color depth, position of the central figure, and so on), and each point is analyzed. Human images have certain types of shapes and pixels in certain locations (which correspond to the position of the face, legs, mouth, and so on). Each new image is compared to the set of known “good images,” and a deviation is calculated to determine how different the new image is from the average human

image and, therefore, the probability that what is shown is a human figure. This process is called *classification*.

After training, the machine should be able to recognize human shapes. Before real field deployments, the machine is usually tested with unlabeled pictures—this is called the validation or the test set, depending on the ML system used—to verify that the recognition level is at acceptable thresholds. If the machine does not reach the level of success expected, more training is needed.

In other cases, the learning process is not about classifying in two or more categories but about finding a correct value. For example, the speed of the flow of oil in a pipe is a function of the size of the pipe, the viscosity of the oil, pressure, and a few other factors. When you train the machine with measured values, the machine can predict the speed of the flow for a new, and unmeasured, viscosity. This process is called *regression*; regression predicts numeric values, whereas classification predicts categories.

Unsupervised Learning

In some cases, supervised learning is not the best method for a machine to help with a human decision. Suppose that you are processing IoT data from a factory manufacturing small engines. You know that about 0.1% of the produced engines on average need adjustments to prevent later defects, and your task is to identify them before they get mounted into machines and shipped away from the factory. With hundreds of parts, it may be very difficult to detect the potential defects, and it is almost impossible to train a machine to recognize issues that may not be visible. However, you can test each engine and record multiple parameters, such as sound, pressure, temperature of key parts, and so on. Once data is recorded, you can graph these elements in relation to one another (for example, temperature as a function of pressure, sound versus rotating speed over time). You can then input this data into a computer and use mathematical functions to find groups. For example, you may decide to group the engines by the sound they make at a given temperature. A standard function to operate this grouping, *K-means clustering*, finds the mean values for a group of engines (for example, mean value for temperature, mean frequency for sound). Grouping the engines this way can quickly reveal several types of engines that all belong to the same category (for example, small engine of chainsaw type, medium engine of lawnmower type). All engines of the same type produce sounds and temperatures in the same range as the other members of the same group.

There will occasionally be an engine in the group that displays unusual characteristics (slightly out of expected temperature or sound range). This is the engine that you send for manual evaluation. The computing process associated with this determination is called *unsupervised learning*. This type of learning is unsupervised because there is not a “good” or “bad” answer known in advance. It is the variation from a group behavior that allows the computer to learn that something is different. The example of engines is, of course, very simple. In most cases, parameters are multidimensional. In other words, hundreds or thousands of parameters are

computed, and small cumulated deviations in multiple dimensions are used to identify the exception. [Figure 7-5](#) shows an example of such grouping and deviation identification logic. Three parameters are graphed (components 1, 2, and 3), and four distinct groups (clusters) are found. You can see some points that are far from the respective groups. Individual devices that display such “out of cluster” characteristics should be examined more closely individually.

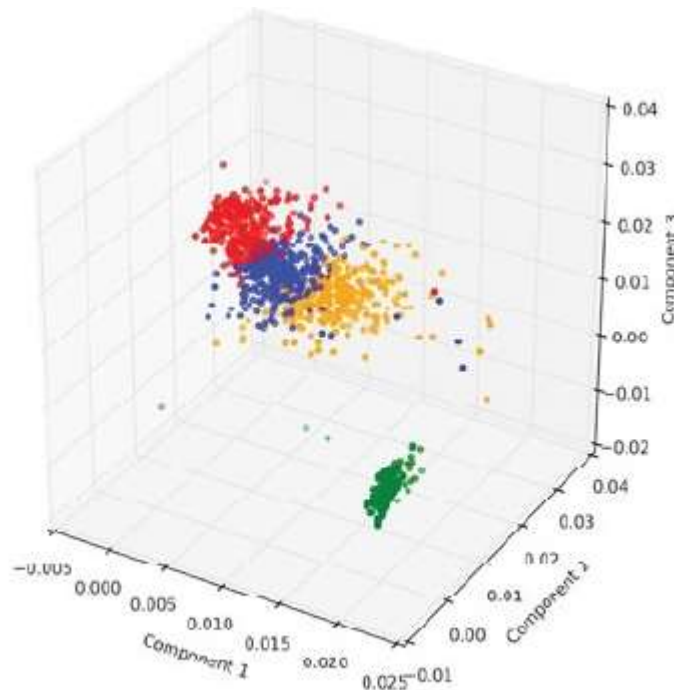


Figure 7-5 *Clustering and Deviation Detection Example*

Neural Networks

Processing multiple dimensions requires a lot of computing power. It is also difficult to determine what parameters to input and what combined variations should raise red flags. Similarly, supervised learning is efficient only with a large training set; larger training sets usually lead to higher accuracy in the prediction. This requirement is partly what made ML fade away somewhat in the 1980s and 1990s. Training the machines was often deemed too expensive and complicated.

Since the 2000s, cheaper computing power along with access to very large data sets (shared over the Internet) rejuvenated the possibilities of ML. At the same time, immense progress has been made in the efficiency of the algorithms used. Take the case of the human shape recognition for mining operations. Distinguishing between a human and a car is easy. The computer can recognize that humans have distinct shapes (such as legs or arms) and that vehicles do not. Distinguishing a human from another mammal is much more difficult (although nonhuman mammals are not common occurrences in mines). The same goes for telling the difference between a pickup truck and a van. You can tell when you see one, but training a machine to differentiate them requires more than basic shape recognition.

This is where neural networks come into the picture. Neural networks are ML methods that mimic the way the human brain works. When you look at a human figure, multiple zones of your brain are activated to recognize colors, movements, facial expressions, and so on. Your brain combines these

elements to conclude that the shape you are seeing is human. Neural networks mimic the same logic. The information goes through different algorithms (called units), each of which is in charge of processing an aspect of the information. The resulting value of one unit computation can be used directly or fed into another unit for further processing to occur. In this case, the neural network is said to have several layers. For example, a neural network processing human image recognition may have two units in a first layer that determines whether the image has straight lines and sharp angles—because vehicles commonly have straight lines and sharp angles, and human figures do not. If the image passes the first layer successfully (because there are no or only a small percentage of sharp angles and straight lines), a second layer may look for different features (presence of face, arms, and so on), and then a third layer might compare the image to images of various animals and conclude that the shape is a human (or not). The great efficiency of neural networks is that each unit processes a simple test, and therefore computation is quite fast. This model is demonstrated in Figure 7-6

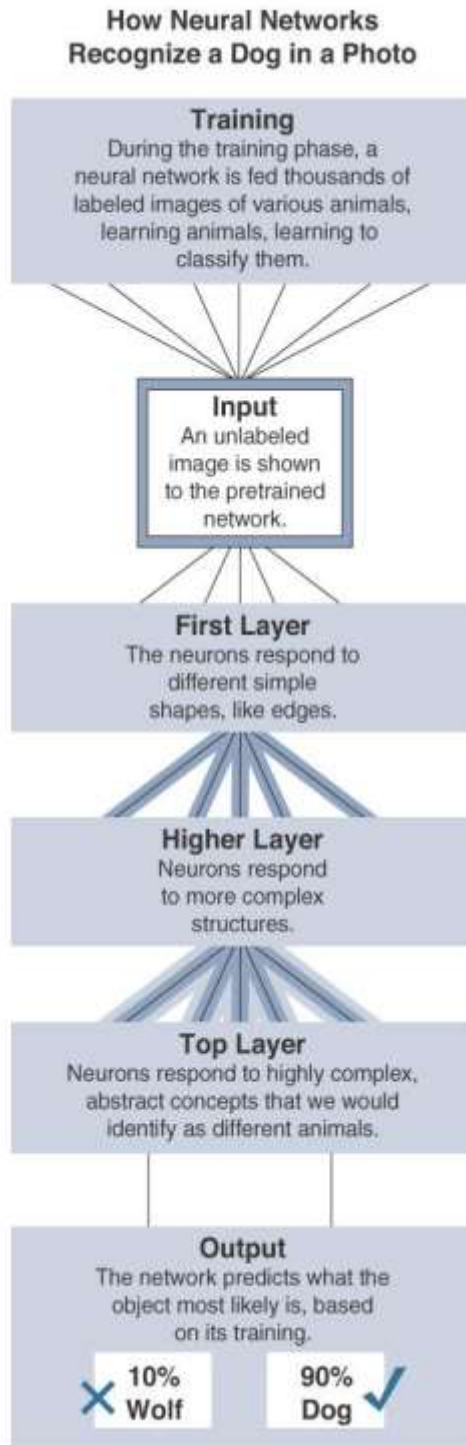


Figure 7-6 Neural Network Example

By contrast, old supervised ML techniques would compare the human figure to potentially hundreds of thousands of images during the training phase, pixel by pixel, making them difficult and expensive to implement (with a lot of training needed) and slow to operate. Neural networks have been the subject of much research work. Multiple research and optimization efforts have examined the number of units

and layers, the type of data processed at each layer, and the type and combination of algorithms used to process the data to make processing more efficient for specific applications. Image processing can be optimized with certain types of algorithms that may not be optimal for crowd movement classification. Another algorithm may be found in this case that would revolutionize the way these movements are processed and analyzed. Possibilities are as numerous as the applications where they can be used.

In a sense, neural networks rely on the idea that information is divided into key components, and each component is assigned a weight. The weights compared together decide the classification of this information (no straight lines + face + smile = human).

When the result of a layer is fed into another layer, the process is called deep learning (“deep” because the learning process has more than a single layer). One advantage of deep learning is that having more layers allows for richer intermediate processing and representation of the data. At each layer, the data can be formatted to be better utilized by the next layer. This process increases the efficiency of the overall result.

Machine Learning and Getting Intelligence from Big Data

When the principles of machine learning are clear, the application to IoT becomes obvious. The difficulty resides in determining the right algorithm and the right learning model for each use case. Such an analysis goes beyond the scope of this chapter, but it can be useful to organize ML operations into two broad subgroups:

- **Local learning:** In this group, data is collected and processed locally, either in the sensor itself (the edge node) or in the gateway (the fog node).
- **Remote learning:** In this group, data is collected and sent to a central computing unit (typically the data center in a specific location or in the cloud), where it is processed.

Note Associated with these two subgroups, you will encounter the term inherited learning. This term refers to results of learning that the local unit received from elsewhere. For example, a processing computer may collect data from multiple sensors and gateways, perform ML on this data, and send the resulting behavioral change request or conclusion back to the gateway and the sensor. This new received knowledge optimizes local operations and is inherited learning (as opposed to simple local learning).

Regardless of the location where (and, therefore, the scale at which) data is processed, common applications of ML for IoT revolve around four major domains:

- **Monitoring:** Smart objects monitor the environment where they operate. Data is processed to better understand the conditions of operations. These conditions can refer to external factors, such as air temperature, humidity, or presence of carbon dioxide in a mine, or to operational internal factors, such as the pressure of a pump, the viscosity of oil flowing in a pipe, and so on. ML can be used with monitoring to detect early failure conditions (for example, K-means deviations showing out-of-range behavior) or to better evaluate the environment (such as shape recognition for a robot automatically sorting material or picking goods in a warehouse or a supply chain).
- **Behavior control:** Monitoring commonly works in conjunction with behavior control. When a given set of parameters reach a target threshold—defined in advance (that is, supervised) or

learned dynamically through deviation from mean values (that is, unsupervised) —monitoring functions generate an alarm. This alarm can be relayed to a human, but a more efficient and more advanced system would trigger a corrective action, such as increasing the flow of fresh air in the mine tunnel, turning the robot arm, or reducing the oil pressure in the pipe.

- **Operations optimization:** Behavior control typically aims at taking corrective actions based on thresholds. However, analyzing data can also lead to changes that improve the overall process. For example, a water purification plant in a smart city can implement a system to monitor the efficiency of the purification process based on which chemical (from company A or company B) is used, at what temperature, and associated to what stirring mechanism (stirring speed and depth). Neural networks can combine multiples of such units, in one or several layers, to estimate the best chemical and stirring mix for a target air temperature. This intelligence can help the plant reduce its consumption of chemicals while still operating at the same purification efficiency level. As a result of the learning, behavior control results in different machine actions. The objective is not merely to pilot the operations but to improve the efficiency and the result of these operations.
- **Self-healing, self-optimizing:** A fast-developing aspect of deep learning is the closed loop. ML-based monitoring triggers changes in machine behavior (the change is monitored by humans), and operations optimizations. In turn, the ML engine can be programmed to dynamically monitor and combine new parameters (randomly or semi-randomly) and automatically deduce and implement new optimizations when the results demonstrate a possible gain. The system becomes self-learning and self-optimizing. It also detects new K-means deviations that result in predetection of new potential defects, allowing the system to self-heal. The healing is not literal, as external factors (typically human operators) have to intervene, but the diagnosis is automated. In many cases, the system can also automatically order a piece of equipment that is detected as being close to failure or automatically take corrective actions to avoid the failure (for example, slow down operations, modify a machine's movement to avoid fatigue on a weak link).

For all these operations, a specific aspect of ML for IoT is the scale. A weather sensor mounted on a light pole in a street can provide information about the local pollution level. At the scale of the entire city, the authorities can monitor moving pollution clouds, and the global and local effects of mist or humidity, pressure, and terrain. All this information can be combined with traffic data to globally regulate traffic light patterns, reduce emissions from industrial pollution sources, or increase the density of mass transit vehicles along the more affected axes. Meanwhile, at the local level, the LED on the light pole can increase or reduce its luminosity and change its color to adapt to local conditions. This change can be driven by either local condition processing (local learning) or inherited learning.

The ability to combine fog computing on specific and specialized systems with cloud computing on data coming from multiple sources and derive global or local corrective actions is what makes ML so powerful for IoT. With open systems and the explosion of smart objects, the possibilities of correlations and cross-optimizations are very wide.

Predictive Analytics

Machine learning and big data processing for IoT fit very well into the digitization described in Chapter 1, “What Is IoT?” The advanced stages of this model see the network self-diagnose and self-optimize. In

the IoT world, this behavior is what the previous section describes. When data from multiple systems is combined and analyzed together, predictions can be made about the state of the system. For example, Chapter 13, “Transportation,” examines the case of sensors deployed on locomotives. Multiple smart objects measure the pull between carriages, the weight on each wheel, and multiple other parameters to offer a form of cruise control optimization for the driver. At the same time, cameras observe the state of the tracks ahead, audio sensors analyze the sound of each wheel on the tracks, and multiple engine parameters are measured and analyzed. All this data can be returned to a data processing center in the cloud that can re-create a virtual twin of each locomotive. Modeling the state of each locomotive and combining this knowledge with anticipated travel and with the states (and detected failures) of all other locomotives of the same type circulating on the tracks of the entire city, province, state, or country allows the analytics platform to make very accurate predictions on what issue is likely to affect each train and each locomotive. Such predictive analysis allows preemptive maintenance and increases the safety and efficiency of operations.

Similarly, sensors combined with big data can anticipate defects or issues in vehicles operating in mines, in manufacturing machines, or any system that can be monitored, along with other similar systems.

Big Data Analytics Tools and Technology

It is a common mistake for individuals new to the world of data management to use the terms big data and Hadoop interchangeably. Though it’s true that Hadoop is at the core of many of today’s big data implementations, it’s not the only piece of the puzzle. Big data analytics can consist of many different software pieces that together collect, store, manipulate, and analyze all different data types. It helps to better understand the landscape by defining what big data is and what it is not. Generally, the industry looks to the “three Vs” to categorize big data:

- **Velocity:** Velocity refers to how quickly data is being collected and analyzed. Hadoop Distributed File System is designed to ingest and process data very quickly. Smart objects can generate machine and sensor data at a very fast rate and require database or file systems capable of equally fast ingest functions.
- **Variety:** Variety refers to different types of data. Often you see data categorized as structured, semi-structured, or unstructured. Different database technologies may only be capable of accepting one of these types. Hadoop is able to collect and store all three types. This can be beneficial when combining machine data from IoT devices that is very structured in nature with data from other sources, such as social media or multimedia, that is unstructured.
- **Volume:** Volume refers to the scale of the data. Typically, this is measured from gigabytes on the very low end to petabytes or even exabytes of data on the other extreme. Generally, big data implementations scale beyond what is available on locally attached storage disks on a single node. It is common to see clusters of servers that consist of dozens, hundreds, or even thousands of nodes for some large deployments.

The characteristics of big data can be defined by the sources and types of data. First is machine data, which is generated by IoT devices and is typically unstructured data. Second is transactional data, which is from sources that produce data from transactions on these systems, and, have high volume and structured. Third is social data sources, which are typically high volume and structured. Fourth is

enterprise data, which is data that is lower in volume and very structured. Hence big data consists of data from all these separate sources.

An additional point to consider while reviewing data sources is the amount of data ingested from each source, which determines the data storage layer design. You should also consider the mechanism to get the data from the ingest systems—namely push or pull. The type of data source—database, file, web service, stream—also needs to be considered as it also determines the structure of data.

Data ingest is the layer that connects data sources to storage. It's the layer that preprocesses, validates, extracts, and stores data temporarily for further processing. There are several patterns to consider for data ingest. First is multisource ingestion, which connects multiple data sources to ingest systems. In this pattern, ingest nodes receive streams of data from multiple sources and do processing before passing the data to intermediate nodes and to final store nodes. This pattern is typically implemented in batch systems and (less often, due to the delay of data availability) in real-time systems.

Data collection and analysis are not new concepts in the industries that helped define IoT. Industrial verticals have long depended on the ability to get, collect, and record data from various processes in order to record trends and track performance and quality.

For example, many industrial automation and control systems feed data into two distinct database types, relational databases and historians. Relational databases, such as Oracle and Microsoft SQL, are good for transactional, or process, data. Their benefit is being able to analyze complex data relationships on data that arrives over a period of time. On the other hand, historians are optimized for time-series data from systems and processes. They are built with speed of storage and retrieval of data at their core, recording each data point in a series with the pertinent information about the system being logged. This data may consist of a sensor reading, the quantity of a material, a temperature reading, or flow data.

Relational databases and historians are mature technologies that have been with us for many years, but new technologies and techniques in the data management market have opened up new possibilities for sensor and machine data. These database technologies broadly fit into a few categories that each have strengths and potential drawbacks when used in an IoT context. The three most popular of these categories are massively parallel processing systems, NoSQL, and Hadoop.

Massively Parallel Processing Databases

Enterprises have used relational databases for storing structured, row and column style data types for decades. Relational databases are often grouped into a broad data storage category called data warehouses. Though they are the centerpiece of most data architectures, they are often used for longer-term archiving and data queries that can often take minutes or hours. An example of this would be asking for all the items produced in the past year that had a particular specification. Depending on the number of items in the database and the complexity of the question being asked, the response could be slow to return.

Massively parallel processing (MPP) databases were built on the concept of the relational data warehouses but are designed to be much faster, to be efficient, and to support reduced query times. To accomplish this, MPP databases take advantage of multiple nodes (computers) designed in a scale-out architecture such that both data and processing are distributed across multiple systems.

MPPs are sometimes referred to as analytic databases because they are designed to allow for fast query processing and often have built-in analytic functions. As the name implies, these database types process massive data sets in parallel across many processors and nodes. An MPP architecture (see Figure 7-7) typically contains a single master node that is responsible for the coordination of all the data storage and processing across the cluster. It operates in a “shared-nothing” fashion, with each node containing local processing, memory, and storage and operating independently. Data storage is optimized across the nodes in a structured SQL-like format that allows data analysts to work with the data using common SQL tools and applications. The earlier example of a complex SQL query could be distributed and optimized, resulting in a significantly faster response. Because data stored on MPPs must still conform to this relational structure, it may not be the only database type used in an IoT implementation. The sources and types of data may vary, requiring a database that is more flexible than relational databases allow.

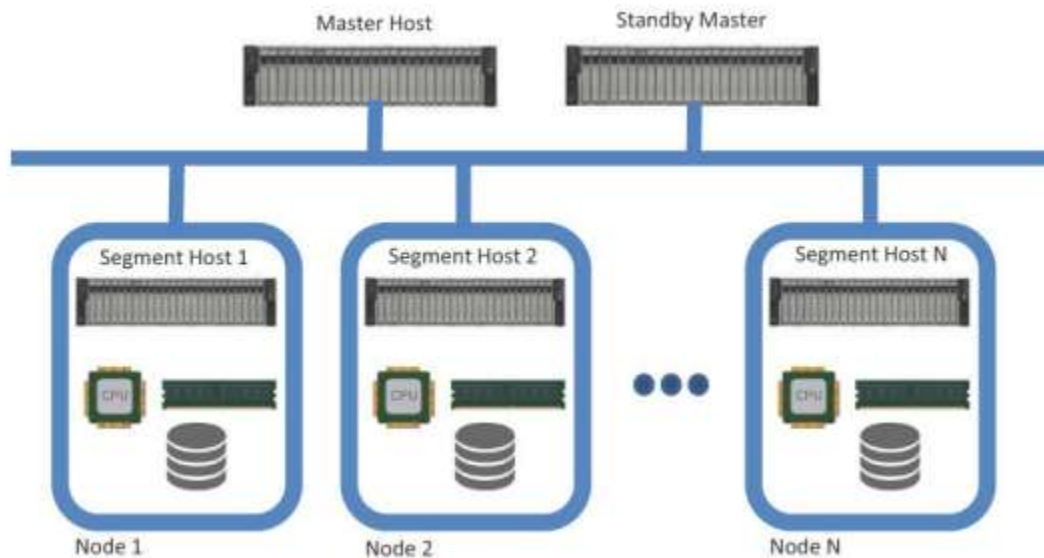


Figure 7-7 *MPP Shared-Nothing Architecture*

NoSQL Databases

NoSQL (“not only SQL”) is a class of databases that support semi-structured and unstructured data, in addition to the structured data handled by data warehouses and MPPs. NoSQL is not a specific database technology; rather, it is an umbrella term that encompasses several different types of databases, including the following:

- **Document stores:** This type of database stores semi-structured data, such as XML or JSON. Document stores generally have query engines and indexing features that allow for many optimized queries.

- **Key-value stores:** This type of database stores associative arrays where a key is paired with an associated value. These databases are easy to build and easy to scale.
- **Wide-column stores:** This type of database stores similar to a key-value store, but the formatting of the values can vary from row to row, even in the same table.
- **Graph stores:** This type of database is organized based on the relationships between elements. Graph stores are commonly used for social media or natural language processing, where the connections between data are very relevant.

NoSQL was developed to support the high-velocity, urgent data requirements of modern web applications that typically do not require much repeated use. The original intent was to quickly ingest rapidly changing server logs and clickstream data generated by web-scale applications that did not neatly fit into the rows and columns required by relational databases. Similar to other data stores, like MPPs and Hadoop (discussed later), NoSQL is built to scale horizontally, allowing the database to span multiple hosts, and can even be distributed geographically.

Expanding NoSQL databases to other nodes is similar to expansion in other distributed data systems, where additional hosts are managed by a master node or process. This expansion can be automated by some NoSQL implementations or can be provisioned manually. This level of flexibility makes NoSQL a good candidate for holding machine and sensor data associated with smart objects.

Of the database types that fit under the NoSQL category, key-value stores and document stores tend to be the best fit for what is considered “IoT data.” Key-value store is the technology that provides the foundation for many of today’s RDBMSs, such as MS SQL, Oracle, and DB2.³ However, unlike traditional RDBMSs, key-value stores on NoSQL are not limited to a single monolithic system. NoSQL key-value stores are capable of handling indexing and persistence simultaneously at a high rate. This makes it a great choice for time-series data sets, which record a value at a given interval of time, such as a temperature or pressure reading from a sensor.

By allowing the database schema to change quickly, NoSQL document databases tend to be more flexible than key-value store databases. Semi-structured or unstructured data that does not neatly fit into rows and columns can share the same database with organized time-series data. Unstructured data can take many forms; two examples are a photograph of a finished good on a manufacturing line used for QA and a maintenance report from a piece of equipment.

Many NoSQL databases provide additional capabilities, such as being able to query and analyze data within the database itself, eliminating the need to move and process it elsewhere. They also provide a variety of ways to query the database through an API, making it easy to integrate them with other data management applications.

Hadoop

Hadoop is the most recent entrant into the data management market, but it is arguably the most popular choice as a data repository and processing engine. Hadoop was originally developed as a result of projects at Google and Yahoo!, and the original intent for Hadoop was to index millions of websites and quickly return search results for open source search engines. Initially, the project had two key elements:

- **Hadoop Distributed File System (HDFS):** A system for storing data across multiple nodes
- **MapReduce:** A distributed processing engine that splits a large task into smaller ones that can be run in parallel.

Both of these elements are still present in current Hadoop distributions and provide the foundation for other projects that are discussed later in this chapter.

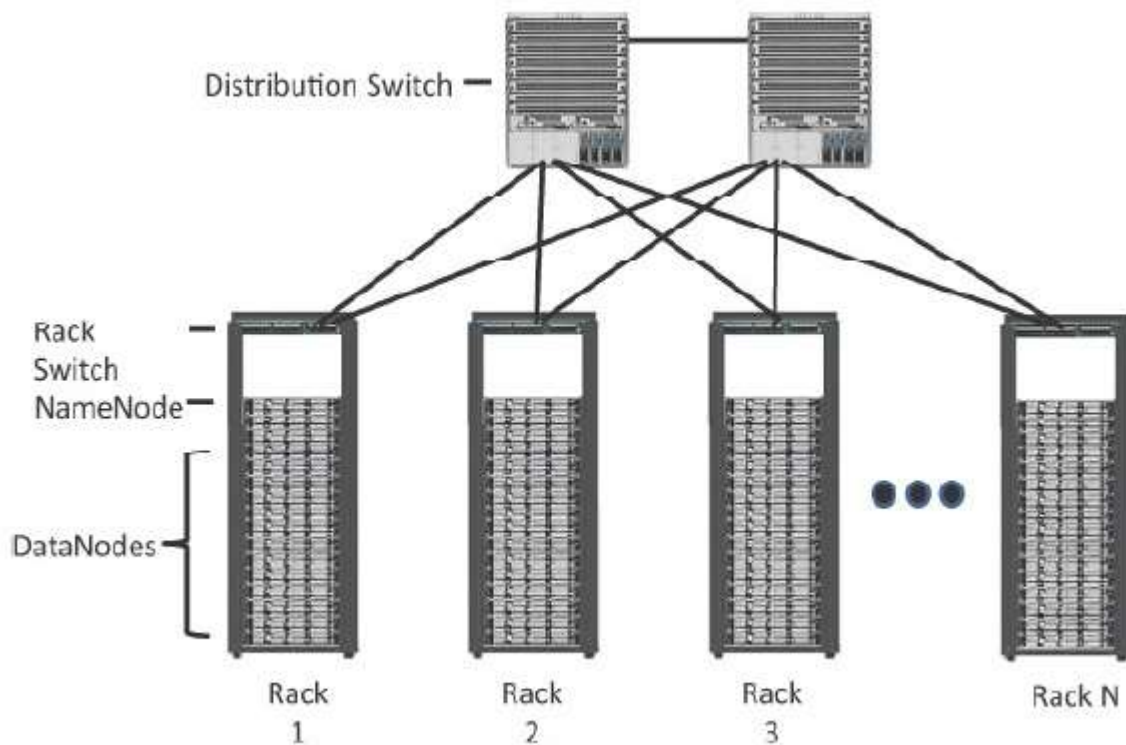


Figure 7-8 *Distributed Hadoop Cluster*

Much like the MPP and NoSQL systems discussed earlier, Hadoop relies on a scale-out architecture that leverages local processing, memory, and storage to distribute tasks and provide a scalable storage system for data. Both MapReduce and HDFS take advantage of this distributed architecture to store and process massive amounts of data and are thus able to leverage resources from all nodes in the cluster. For HDFS, this capability is handled by specialized nodes in the cluster, including NameNodes and DataNodes (see [Figure 7-8](#)):

- **NameNodes:** These are a critical piece in data adds, moves, deletes, and reads on HDFS. They coordinate where the data is stored, and maintain a map of where each block of data is stored and where it is replicated. All interaction with HDFS is coordinated through the primary (active) NameNode, with a secondary (standby) NameNode notified of the changes in the event of a failure of the primary. The NameNode takes write requests from clients and distributes those files across the available nodes in configurable block sizes, usually 64 MB or 128 MB blocks. The NameNode is also responsible for instructing the DataNodes where replication should occur.
- **DataNodes:** These are the servers where the data is stored at the direction of the NameNode. It is common to have many DataNodes in a Hadoop cluster to store the data. Data blocks are distributed across several nodes and often are replicated three, four, or more times across nodes for redundancy. Once data is written to one of the DataNodes, the DataNode selects two (or more) additional nodes, based on replication policies, to ensure data redundancy across the cluster. Disk redundancy techniques such as Redundant Array of Independent Disks (RAID) are generally not used for HDFS because the NameNodes and DataNodes coordinate block-level redundancy with this replication technique.

Figure 7-9 shows the relationship between NameNodes and DataNodes and how data blocks are distributed across the cluster.

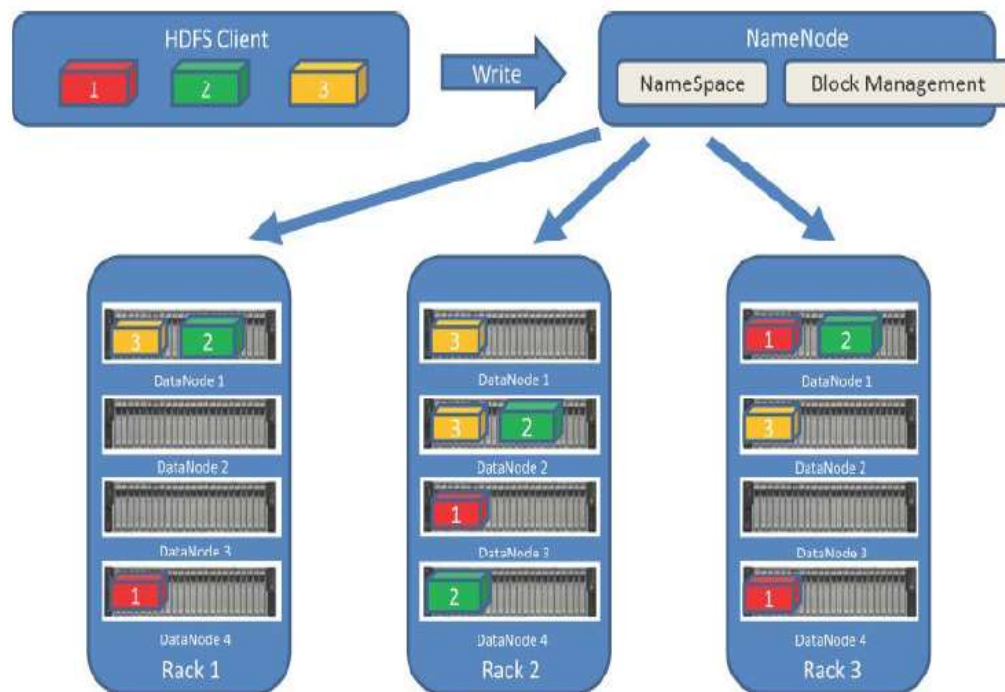


Figure 7-9 Writing a File to HDFS

MapReduce leverages a similar model to batch process the data stored on the cluster nodes. Batch processing is the process of running a scheduled or ad hoc query across historical data stored in the HDFS. A query is broken down into smaller tasks and distributed across all the nodes running MapReduce in a cluster. While this is useful for understanding patterns and trending in historical sensor or machine data, it has one significant drawback: time. Depending on how much data is being queried and the complexity of the query, the result could take seconds or minutes to return. If you have a real-time process running where you need a result at a moment's notice, MapReduce is not the right data processing engine for that.

YARN

Introduced with version 2.0 of Hadoop, YARN (Yet Another Resource Negotiator) was designed to enhance the functionality of MapReduce. With the initial release, MapReduce was responsible for batch data processing and job tracking and resource management across the cluster. YARN was developed to take over the resource negotiation and job/task tracking, allowing MapReduce to be responsible only for data processing.

With the development of a dedicated cluster resource scheduler, Hadoop was able to add additional data processing modules to its core feature set, including interactive SQL and real-time processing, in addition to batch processing using MapReduce.

The Hadoop Ecosystem

As mentioned earlier, Hadoop plays an increasingly big role in the collection, storage, and processing of IoT data due to its highly scalable nature and its ability to work with large volumes of data. Many organizations have adopted Hadoop clusters for storage and processing of data and have looked for complimentary software packages to add additional functionality to their distributed Hadoop clusters. Since the initial release of Hadoop in 2011, many projects have been developed to add incremental functionality to Hadoop and have collectively become known as the *Hadoop ecosystem*.

Hadoop may have had meager beginnings as a system for distributed storage and processing, but it has since grown into a robust collection of projects that, combined, create a very complete data management and analytics framework. Hadoop now comprises more than 100 software projects under the Hadoop umbrella, capable of nearly every element in the data lifecycle, from collection, to storage, to processing, to analysis and visualization. Each of these individual projects is a unique piece of the overall data management solution. The following sections describe several of these packages and discuss how they are used to collect or process data.

Apache Kafka

Part of processing real-time events, such as those commonly generated by smart objects, is having them ingested into a processing engine. The process of collecting data from a sensor or log file and preparing it to be processed and analyzed is typically handled by messaging systems. Messaging systems are designed to accept data, or messages, from where the data is

generated and deliver the data to stream-processing engines such as Spark Streaming or Storm. Apache Kafka is a distributed publisher-subscriber messaging system that is built to be scalable and fast. It is composed of topics, or message brokers, where producers write data and consumers read data from these topics. Figure 7-10 shows the data flow from the smart objects (producers), through a topic in Kafka, to the real-time processing engine. Due to the distributed nature of Kafka, it can run in a clustered configuration that can handle many producers and consumers simultaneously and exchanges information between nodes, allowing topics to be distributed over multiple nodes. The goal of Kafka is to provide a simple way to connect to data sources and allow consumers to connect to that data in the way they would like. The following sections describe several of these packages and discusses how they are used to collect or process data.

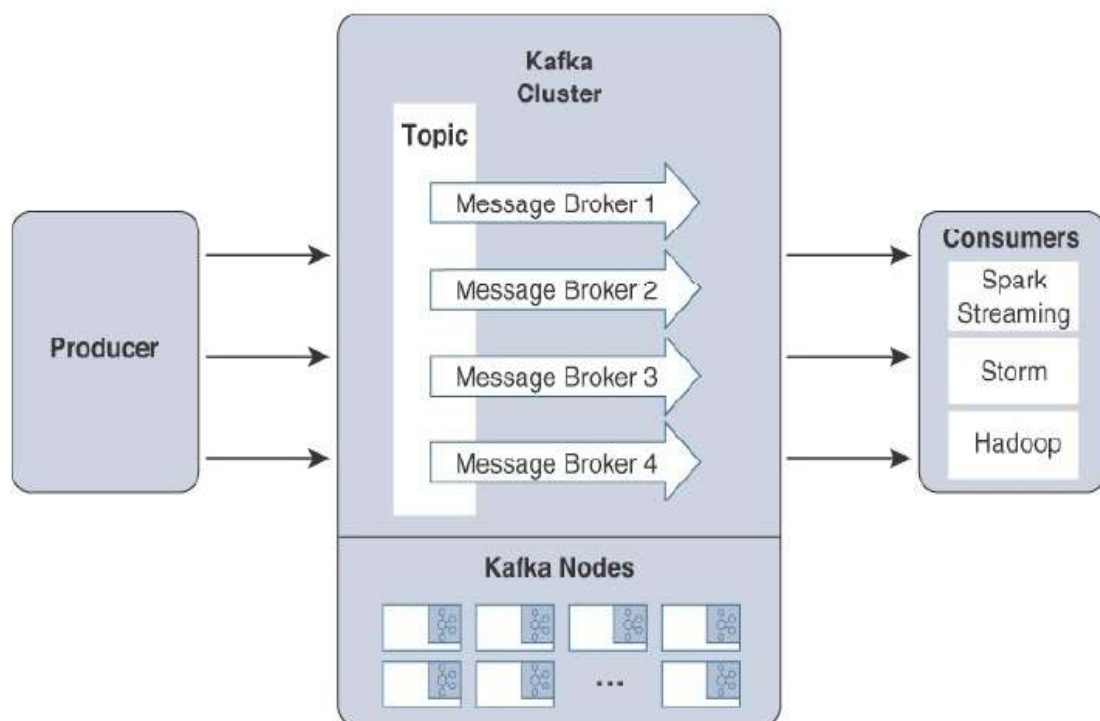


Figure 7-10 *Apache Kafka Data Flow*

Apache Spark

Apache Spark is an in-memory distributed data analytics platform designed to accelerate processes in the Hadoop ecosystem. The “in-memory” characteristic of Spark is what enables it to run jobs very quickly. At each stage of a MapReduce operation, the data is read and written back to the disk, which means latency is introduced through each disk operation. However, with Spark, the processing of this data is moved into high-speed memory, which has significantly lower latency. This speeds the batch processing jobs and also allows for near-real-time processing of events.

Real-time processing is done by a component of the Apache Spark project called Spark Streaming. Spark Streaming is an extension of Spark Core that is responsible for taking live streamed data from a messaging system, like Kafka, and dividing it into smaller microbatches. These microbatches are called discretized streams, or DStreams. The Spark processing engine is able to operate on these smaller pieces of data, allowing rapid insights into the data and subsequent actions. Due to this “instant feedback” capability, Spark is becoming an important component in many IoT deployments. Systems that control safety and security of personnel, time-sensitive processes in the manufacturing space, and infrastructure control in traffic management all benefit from these real-time streaming capabilities.

Edge Streaming Analytics

A major area of evolution for IT in the past few years has been the transition to cloud services. Nearly every large technology company is now selling software and services from the cloud, and this includes data analytics systems, whether they are offered as a service from a public cloud operator or are built in massive private data center clouds. However, analyzing a massive volume of time-sensitive IoT data in a centralized cloud is often not ideal. In the world of IoT, vast quantities of data are generated on the fly and often need to be analyzed and responded to immediately. Not only is the volume of data generated at the edge immense—meaning the bandwidth requirements to the cloud or data center need to be engineered to match—but the data may be so time sensitive that it needs immediate attention, and waiting for deep analysis in the cloud simply isn’t possible.

One industry where data analytics is used extensively is the world of automobile racing. For example, in Formula One racing, each car has between 150 to 200 sensors that, combined, generate more than 1000 data points per second, resulting in hundreds of gigabytes of raw data per race. The sensor data is transmitted from the car and picked up by track-side wireless sensors. During a race, weather conditions may vary, tire conditions change, and accidents or other racing incidents almost always require an adaptable and flexible racing strategy. As the race develops, decisions such as when to pit, what tires to use, when to pass, and when to slow down all need to be made in seconds. Teams have found that enormous insights leading to better race results can be gained by analyzing data on the fly—and the data may come from many different sources, including trackside sensors, car telemetry, and weather reports.

Most teams use sophisticated data analytics systems to enhance racing strategy, but in many cases, this equipment resides back in the team’s data center, far away from the track. For a team that has its analytics software in a data center in the UK, the latency to Australia (the most remote race) is several hundred milliseconds away. The time it takes to collect and analyze this data as a batch process in a distant part of the world is not only inefficient but can mean the difference between a successful race strategy that adapts to changing conditions and one that lacks the flexibility and agility to send meaningful instructions to the drivers. In short, it can mean the difference between winning and losing a race.

Comparing Big Data and Edge Analytics

When you hear the term *big data*, it is usually in reference to unstructured data that has been collected and stored in the cloud. The data is collected over time so that it can be analyzed through batch processing tools, such as an RDBMS, Hadoop, or some other tool, at which point business insights are gained, and value is drawn from the data. Tools like Hadoop and MapReduce are great at tackling problems that require deep analytics on a large and complex quantity of unstructured data; however, due to their distance from the IoT endpoints and the bandwidth required to bring all the data back to the cloud, they are generally not well suited to real-time analysis of data as it is generated.

In applying data analytics to the car racing example discussed earlier, big data analytics is used to examine all the statistics of the racing team and players based on their performance in the data center or cloud. While big data can apply analytics in real-time (as discussed earlier), it is mainly focused on batch job analytics on large volumes of data. Streaming analytics involves analyzing a race while it is happening and trying to figure out who is going to win based on the actual performance in real-time—and this analysis is typically performed as close to the edge as possible. Streaming analytics allows you to continually monitor and assess data in real-time so that you can adjust or fine-tune your predictions as the race progresses.

In the context of IoT, with streaming analytics performed at the edge (either at the sensors themselves or very close to them, in a fog node that is, for example, integrated into the gateway), it is possible to process and act on the data in real-time without waiting for the results from a future batch-processing job in the cloud. Does this mean that streaming analytics replaces big data analytics in the cloud? Not at all. They both have roles to play and both contribute to improved business insights and processes.

In one sense, if raw data is generated in the data center, it makes sense to analyze it there. But what if the majority of data is being generated in remote locations by sensors that are spread all over a wide area? To be truly effective at the moment it is created, the data needs to be analyzed and responded to as close to the edge as possible. Once it has been analyzed and reduced at the edge, the resultant data can be sent to the cloud and used to gain deeper insights over time. It is also important to remember that the edge isn't in just one place. The edge is highly distributed, which means analytics at the edge needs to be highly coordinated and structured. This also implies a communications system where edge/fog nodes are able to communicate with each other when necessary and report results to a big data system in the cloud.

From a business perspective, streaming analytics involves acting on data that is generated while it is still valuable, before it becomes stale. For example, roadway sensors combined with GPS way finding apps may tell a driver to avoid a certain highway due to traffic. This data is valuable for only a small window of time. Historically, it may be interesting to see how many traffic accidents or blockages have occurred on a certain segment of highway or to predict congestion

based on past traffic data. However, for the driver in traffic receiving this information, if the data is not acted upon immediately, the data has little value.

From a security perspective, having instantaneous access to analyzed and preprocessed data at the edge also allows an organization to realize anomalies in its network so those anomalies can be quickly contained before spreading to the rest of the network.

To summarize, the key values of edge streaming analytics include the following:

- **Reducing data at the edge:** The aggregate data generated by IoT devices is generally in proportion to the number of devices. The scale of these devices is likely to be huge, and so is the quantity of data they generate. Passing all this data to the cloud is inefficient and is unnecessarily expensive in terms of bandwidth and network infrastructure.
- **Analysis and response at the edge:** Some data is useful only at the edge (such as a factory control feedback system). In cases such as this, the data is best analyzed and acted upon where it is generated.
- **Time sensitivity:** When timely response to data is required, passing data to the cloud for future processing results in unacceptable latency. Edge analytics allows immediate responses to changing conditions.

Edge Analytics Core Functions

To perform analytics at the edge, data needs to be viewed as real-time flows. Whereas big data analytics is focused on large quantities of data at rest, edge analytics continually processes streaming flows of data in motion. Streaming analytics at the edge can be broken down into three simple stages:

- **Raw input data:** This is the raw data coming from the sensors into the analytics processing unit.
- **Analytics processing unit (APU):** The APU filters and combines data streams (or separates the streams, as necessary), organizes them by time windows, and performs various analytical functions. It is at this point that the results may be acted on by micro services running in the APU.
- **Output streams:** The data that is output is organized into insightful streams and is used to influence the behavior of smart objects, and passed on for storage and further processing in the cloud. Communication with the cloud often happens through a standard publisher/subscriber messaging protocol, such as MQTT.

Figure 7-12 illustrates the stages of data processing in an edge APU.

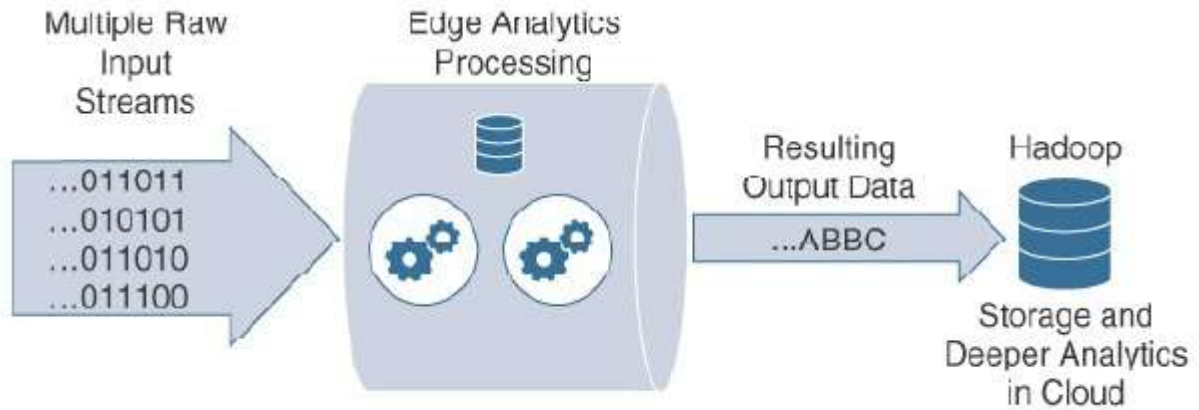


Figure 7-12 *Edge Analytics Processing Unit*

In order to perform analysis in real-time, the APU needs to perform the following functions:

- **Filter:** The streaming data generated by IoT endpoints is likely to be very large, and most of it is irrelevant. For example, a sensor may simply poll on a regular basis to confirm that it is still reachable. This information is not really relevant and can be mostly ignored. The filtering function identifies the information that is considered important.
- **Transform:** In the data warehousing world, Extract, Transform, and Load (ETL) operations are used to manipulate the data structure into a form that can be used for other purposes. Analogous to data warehouse ETL operations, in streaming analytics, once the data is filtered, it needs to be formatted for processing.
- **Time:** As the real-time streaming data flows, a timing context needs to be established. This could be to correlated average temperature readings from sensors on a minute-by-minute basis. For example, [Figure 7-13](#) shows an APU that takes input data from multiple sensors reporting temperature fluctuations. In this case, the APU is programmed to report the average temperature every minute from the sensors, based on an average of the past two minutes. (An example where this may be used is in real-time monitoring of food in a grocery store, where rolling averages of the temperature in open-air refrigeration units needs to be monitored to ensure the safety of the food.) Note that on the left side is the cleaned stream data. This data is presented as streams to the analytics engine (note the syntax at the bottom right of the figure) that establishes the time window and calculates the average temperature over the past two minutes.

Defining Streams and Windows

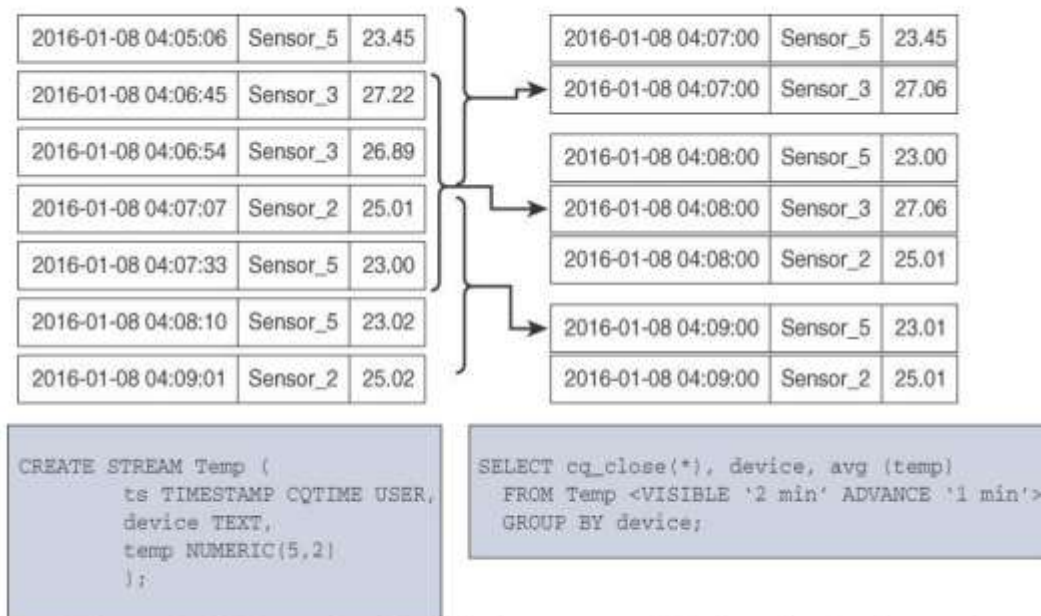


Figure 7-13 Example: Establishing a Time Window for Analytics of Average Temperature from Sensors

- Correlate:** Streaming data analytics becomes most useful when multiple data streams are combined from different types of sensors. For example, in a hospital, several vital signs are measured for patients, including body temperature, blood pressure, heart rate, and respiratory rate. These different types of data come from different instruments, but when this data is combined and analyzed, it provides an invaluable picture of the health of the patient at any given time. 4 However, correlation goes beyond just combining real-time data streams. Another key aspect is combining and correlating real-time measurements with preexisting, or historical, data. For example, historical data may include the patient's past medical history, such as blood test results. Combining historical data gives the live streaming data a powerful context and promotes more insights into the current condition of the patient (see Figure 7-14).

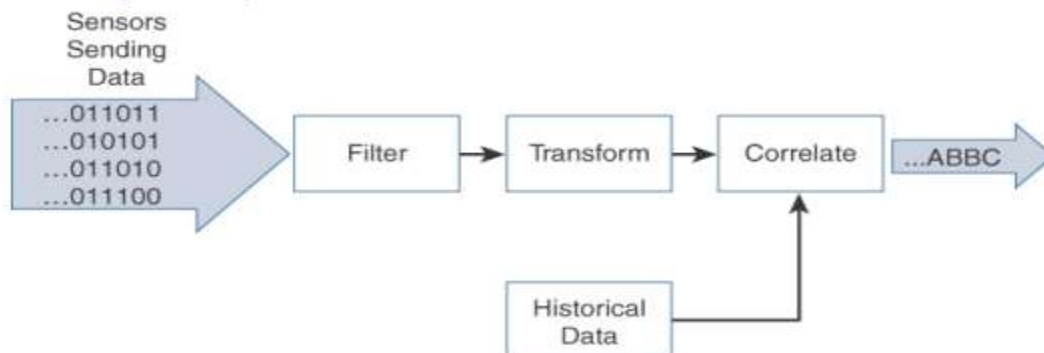


Figure 7-14 Correlating Data Streams with Historical Data

- **Match patterns:** Once the data streams are properly cleaned, transformed, and correlated with other live streams as well as historical data sets, pattern matching operations are used to gain deeper insights to the data. For example, say that the APU has been collecting the patient's vitals for some time and has gained an understanding of the expected patterns for each variable being monitored. If an unexpected event arises, such as a sudden change in heart rate or respiration, the pattern matching operator recognizes this as out of the ordinary and can take certain actions, such as generating an alarm to the nursing staff. The patterns can be simple relationships, or they may be complex, based on the criteria defined by the application. Machine learning may be leveraged to identify these patterns.
- **Improve business intelligence:** Ultimately, the value of edge analytics is in the improvements to business intelligence that were not previously available. For example, conducting edge analytics on patients in a hospital allows staff to respond more quickly to the patient's changing needs and also reduces the volume of unstructured (and not always useful) data sent to the cloud. Over time, the resulting changes in business logic can produce improvements in basic operations, bringing in higher levels of care as well as better efficiencies for the hospital.

Distributed Analytics Systems:

Depending on the application and network architecture, analytics can happen at any point throughout the IoT system. Streaming analytics may be performed directly at the edge, in the fog, or in the cloud data center. There are no hard and-fast rules dictating where analytics should be done, but there are a few guiding principles. We have already discussed the value of reducing the data at the edge, as well as the value of analyzing information so it can be responded to before it gets stale. There is also value in stepping back from the edge to gain a wider view with more data. It's hard to see the forest when you are standing in the middle of it staring at a tree. In other words, sometimes better insights can be gained and data responded to more intelligently when we step back from the edge and look at a wider data set. This is the value of fog computing. (Fog computing is introduced in Chapter 2.) Fog analytics allows you to see beyond one device, giving you visibility into an aggregation of edge nodes and allowing you to correlate data from a wider set. Figure 7-15 shows an example of an oil drilling company that is measuring both pressure and temperature on an oil rig. While there may be some value in doing analytics directly on the edge, in this example, the sensors communicate via MQTT through a message broker to the fog analytics node, allowing a broader data set. (MQTT is discussed in depth in Chapter 6, "Application Protocols for IoT.") The fog node is located on the same oil rig and performs streaming analytics from several edge devices, giving it better insights due to the expanded data set. It may not be able to respond to an event as quickly as analytics performed directly on the edge device, but it is still close to responding in real-time as events occur. Once the fog node is finished with the data, it communicates the results to the cloud (again through a message broker via MQTT) for deeper historical analysis through big data analytics tools

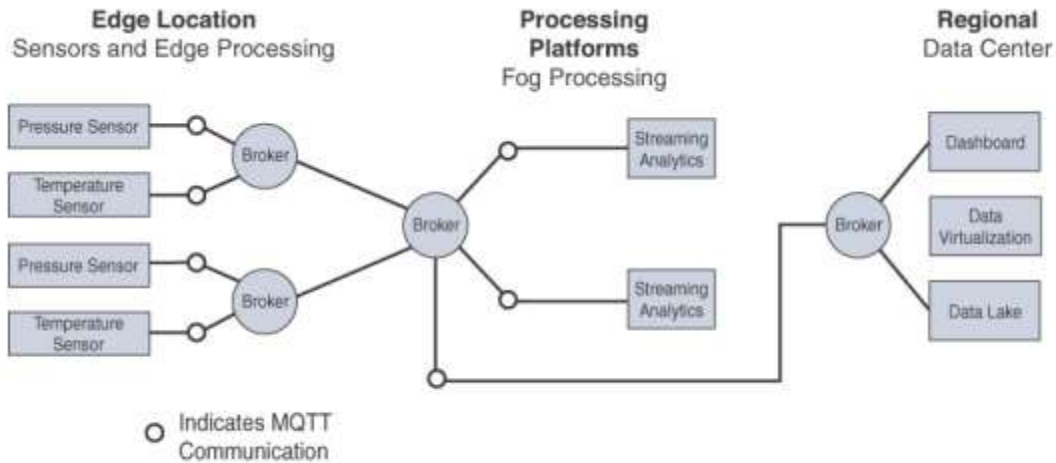


Figure 7-15 *Distributed Analytics Throughout the IoT System*

Network Analytics

Another form of analytics that is extremely important in managing IoT systems is network-based analytics. Unlike the data analytics systems previously discussed that are concerned with finding patterns in the data generated by endpoints, network analytics is concerned with discovering patterns in the communication flows from a network traffic perspective. Network analytics has the power to analyze details of communications patterns made by protocols and correlate this across the network. It allows you to understand what should be considered normal behavior in a network and to quickly identify anomalies that suggest network problems due to suboptimal paths, intrusive malware, or excessive congestion. Analysis of traffic patterns is one of the most powerful tools in an IoT network engineer's troubleshooting arsenal.

As discussed in [Chapter 6](#), IoT endpoints, contrary to generic computing platforms, are designed to directly communicate with a very small number of specific application servers, such as an IoT message or data broker, or specific application servers and network management systems. Therefore, it could be said that IoT solutions and use cases tightly couple devices and applications. [Figure 7-16](#) shows field area network (FAN) traffic analytics performed on the aggregation router in a smart grid.

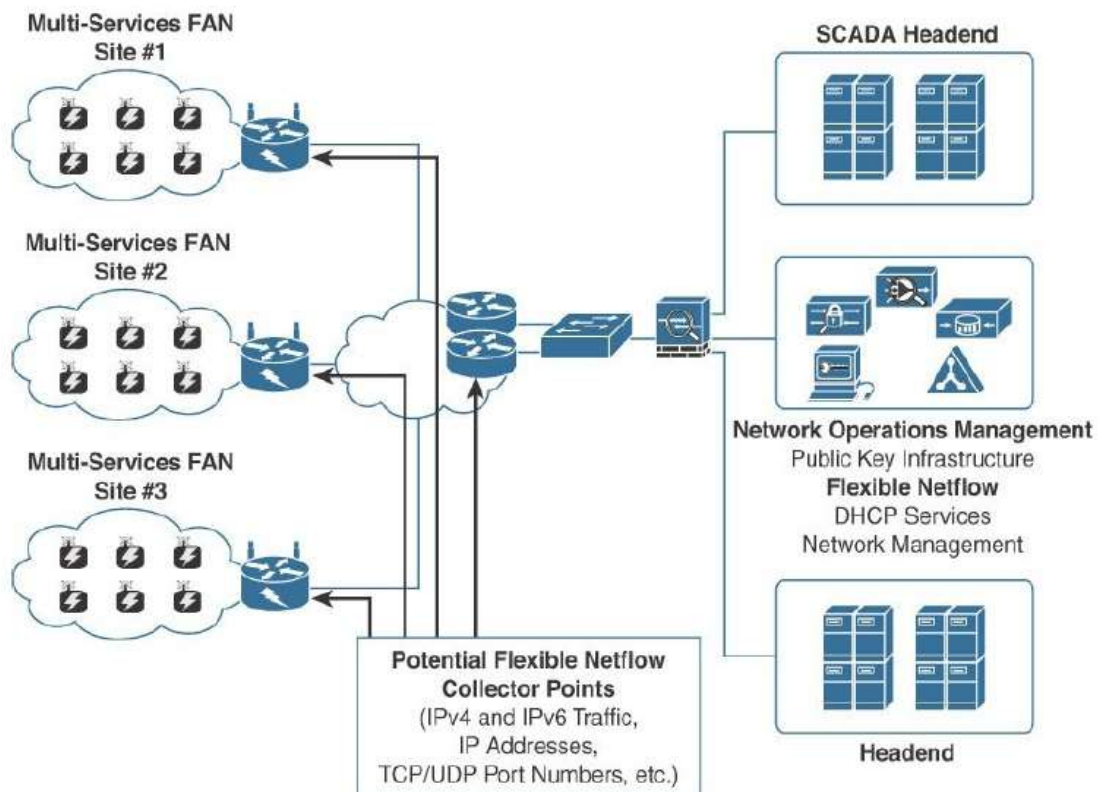


Figure 7-16 Smart Grid FAN Analytics with NetFlow Example

This behavior represents a key aspect that can be leveraged when performing network analytics: Network analytics offer capabilities to cope with capacity planning for scalable IoT deployment as well as security monitoring in order to detect abnormal traffic volume and patterns (such as an unusual traffic spike for a normally quiet protocol) for both centralized or distributed architectures, such as fog computing.

Consider that an IoT device sends its traffic to specific servers, either directly to an application or an IoT broker with the data payload encapsulated in a given protocol. This represents a pair of source and destination addresses, as well as application layer–dependent TCP or UDP port numbers, which can be used for network analytics.

One of the drivers of the adoption of an IP architectural framework for IoT is to leverage tools and processes largely known and deployed by Internet service providers (ISPs) as well as private corporate enterprise networks. To monitor network infrastructure, de facto industry standards and protocols allow pervasive characterization of IP traffic flows, including identification of source and/or destination addresses, data timing and volume, and application types within a network infrastructure. Flow statistics can be collected at different locations in the network. For example, centralized routers or switches that aggregate subnetworks as well as nodes that are highly distributed and connect the last mile of the infrastructure can be used to collect flow information. After data is collected in a known format, it can be sent to an external network

analytics tools that delivers unique services to network managers, like security and performance monitoring and capacity planning.

In the context of IoT infrastructure deployments, for technologies discussed in [Chapter 4](#), “[Connecting Smart Objects](#),” [Chapter 5](#), “[IP as the IoT Network Layer](#),” and [Chapter 6](#), the benefits of flow analytics, in addition to other network management services, are as follows:

- **Network traffic monitoring and profiling:** Flow collection from the network layer provides global and distributed near-real-time monitoring capabilities. IPv4 and IPv6 networkwide traffic volume and pattern analysis helps administrators proactively detect problems and quickly troubleshoot and resolve problems when they occur.
- **Application traffic monitoring and profiling:** Monitoring and profiling can be used to gain a detailed time-based view of IoT access services, such as the application-layer protocols, including MQTT, CoAP, and DNP3, as well as the associated applications that are being used over the network.
- **Capacity planning:** Flow analytics can be used to track and anticipate IoT traffic growth and help in the planning of upgrades when deploying new locations or services by analyzing captured data over a long period of time. This analysis affords the opportunity to track and anticipate IoT network growth on a continual basis.
- **Security analysis:** Because most IoT devices typically generate a low volume of traffic and always send their data to the same server(s), any change in network traffic behavior may indicate a cyber security event, such as a denial of service (DoS) attack. Security can be enforced by ensuring that no traffic is sent outside the scope of the IoT domain. For example, with a LoRaWAN gateway, there should be no reason to see traffic sent or received outside the LoRaWAN network server and network management system. Such traffic could indicate an attack of some sort.
- **Accounting:** In field area networks, routers or gateways are often physically isolated and leverage public cellular services and VPNs for backhaul. Deployments may have thousands of gateways connecting the last-mile IoT infrastructure over a cellular network. Flow monitoring can thus be leveraged to analyze and optimize the billing, in complement with other dedicated applications, such as Cisco Jasper, with a broader scope than just monitoring data flow.
- **Data warehousing and data mining:** Flow data (or derived information) can be warehoused for later retrieval and analysis in support of proactive analysis of multiservice IoT infrastructures and applications.

Xively (formerly known as Cosm and Pachube) was an [Internet of Things](#) (IoT) platform owned by Google. Xively offered product companies a way to connect products, manage connected devices and the data they produce, and integrate that data into other systems. It is pronounced “[zively](#)” (rhymes with lively). Its domain name redirects to a Google page about Cloud IoT Core, though that has been retired.

Xively, now part of Google Cloud, was a platform designed to help companies create and manage Internet of Things (IoT) products. It offered services for securely connecting devices, collecting and managing data, and integrating with other systems for analysis and visualization.

Xively provided features such as device management, data visualization, real-time messaging, and secure connectivity. Users could easily connect devices to the cloud, manage their lifecycle, and analyze data to gain insights and improve their IoT solutions.

However, please note that my training data only goes up to 2022, so I don't have the latest information on Xively's current status or any updates since then.

Xively, now integrated into Google Cloud, was a cloud-based platform designed specifically for the Internet of Things (IoT). It aimed to simplify the development, deployment, and management of IoT solutions by providing a comprehensive set of tools and services.

1. **Device Management:** Xively offered robust device management capabilities, allowing users to easily connect and manage large numbers of devices. This included features such as device provisioning, authentication, and remote configuration.
2. **Data Collection and Storage:** The platform provided tools for collecting, storing, and managing data generated by IoT devices. This included real-time data streaming, historical data storage, and support for various data formats.
3. **Data Visualization and Analytics:** Xively offered tools for visualizing and analyzing IoT data. This included customizable dashboards, real-time data visualization, and integration with analytics tools for advanced data analysis.
4. **Secure Connectivity:** Security was a key focus of Xively, with features such as end-to-end encryption, secure device authentication, and access control to ensure that IoT data and devices were protected from unauthorized access.
5. **Integration and Interoperability:** Xively supported integration with other systems and platforms, allowing users to easily connect their IoT solutions with existing infrastructure and services.
6. **Scalability and Reliability:** Xively was designed to be highly scalable and reliable, capable of handling large-scale IoT deployments with ease.

Overall, Xively aimed to provide a comprehensive and easy-to-use platform for building IoT solutions, enabling businesses to quickly develop and deploy innovative IoT applications.

A Python web application framework is a software framework that is designed to support the development of web applications using the Python programming language. These frameworks provide a structure for building web applications by offering reusable components, libraries, and tools that help developers streamline the development process. Here are some key aspects of Python web application frameworks:

1. **Web Server Gateway Interface (WSGI)**: Python web frameworks typically adhere to the WSGI standard, which defines a simple and universal interface between web servers and Python web applications. This allows different web servers and frameworks to work together seamlessly.
2. **Routing**: Web frameworks provide mechanisms for mapping URLs to specific functions or methods, allowing developers to define the behavior of different parts of the application based on the URL requested by the client.
3. **Templates**: Most Python web frameworks come with a templating engine that allows developers to create HTML templates with placeholders for dynamic content. These templates can then be rendered with data from the application to generate the final HTML output sent to the client.
4. **Database Support**: Python web frameworks often provide built-in support for interacting with databases, making it easier for developers to perform database operations such as querying, inserting, updating, and deleting data.
5. **Security Features**: Web frameworks include features to help developers build secure web applications, such as built-in protection against common security vulnerabilities like cross-site scripting (XSS), cross-site request forgery (CSRF), and SQL injection.
6. **Middleware**: Middleware components provide additional functionality that can be applied to incoming requests or outgoing responses. This can include logging, authentication, and session management.
7. **Development Tools**: Many Python web frameworks come with development tools that help streamline the development process, such as built-in web servers for testing, interactive debugging tools, and automatic code reloading.
8. **Community and Ecosystem**: Python has a vibrant community and a rich ecosystem of third-party libraries and extensions that can be used with web frameworks to add additional functionality, such as authentication, authorization, and API integration.

Some popular Python web frameworks include Django, Flask, and Pyramid. Each framework has its own strengths and weaknesses, so the choice of framework depends on the specific requirements of the web application being developed.

Python is one of the most acceptable languages among web and application developers because of its strong emphasis on efficiency and readability. There are numerous outstanding Python web frameworks, each with their own specialities and features.

Django

Here, we will outline some necessary details and features of Django framework.

Category – Django belongs to the full-stack Python framework.

Release – Latest release – 2.1 version, commonly used release – 1.8, 1.6 version.

About – Built by experienced developers, Django is a high level Python web framework which allows rapid, clean and pragmatic design development. Django handles much of the complexities of web development, so you can focus on writing your app without a need to reinvent the wheel. It's free and open source.

To map objects to database table, Django uses ORM and the same is used to transfer from one database to other. It works with mostly all important databases like Oracle, MySQL, PostgreSQL, SQLite, etc. There are numerous websites in the industry which uses Django as their primary framework for backend development.

Features of Django

Some of the exemplary features of this Python web framework are –

- URL routing
- Authentication
- Database schema migrations
- ORM (Object-relational mapper)
- Template engine

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the "batteries included" philosophy, which means it comes with a set of components and tools to help developers build web applications more quickly and efficiently.

Key features of Django include:

1. **Object-Relational Mapping (ORM):** Django includes a powerful ORM that allows developers to define database models using Python classes, without writing SQL code. This makes it easy to work with databases and perform database operations.
2. **Admin Panel:** Django comes with a built-in admin interface that can be used to manage site content, users, and other aspects of the application. The admin

interface is customizable and can be extended to fit the needs of the application.

3. **URL Routing:** Django uses a URL routing mechanism that maps URL patterns to view functions, allowing developers to create clean and readable URL structures for their applications.
4. **Template System:** Django includes a template system that allows developers to define HTML templates with placeholders for dynamic content. Templates can be used to generate HTML output based on data from the application.
5. **Form Handling:** Django provides a form handling library that simplifies the process of working with forms in web applications. It includes features for form validation, rendering, and processing form data.
6. **Security Features:** Django includes built-in protection against common security vulnerabilities, such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). It also provides tools for managing user authentication and authorization.
7. **Middleware:** Django uses middleware components to process incoming requests and outgoing responses. Middleware can be used to add additional functionality to the application, such as logging, authentication, or caching.
8. **Internationalization and Localization:** Django includes built-in support for internationalization (i18n) and localization (l10n), making it easy to create multilingual web applications.
9. **Testing Framework:** Django includes a testing framework that makes it easy to write and run tests for Django applications. This helps ensure that the application works as expected and allows for easier maintenance and debugging.

Overall, Django is a powerful and flexible web framework that is well-suited for building a wide range of web applications, from simple websites to complex web applications. Its rich feature set, strong community support, and extensive documentation make it a popular choice among developers.

AWS (Amazon Web Services) offers a comprehensive set of services for building and managing IoT (Internet of Things) applications. These services help developers securely connect devices to the cloud, collect and analyze data from those devices, and build applications that interact with them. Here are some key **AWS services for IoT**:

1. **AWS IoT Core:** AWS IoT Core is a managed cloud service that lets connected devices easily and securely interact with cloud applications and other devices. It provides secure communication and data processing between devices and the AWS Cloud.
2. **AWS IoT Device Management:** This service allows you to onboard, organize, monitor, and remotely manage connected devices at scale. It provides features like over-the-air (OTA) updates, remote troubleshooting, and device health monitoring.
3. **AWS IoT Greengrass:** AWS IoT Greengrass extends AWS IoT Core functionality to devices, allowing them to run AWS Lambda functions locally, process data, and communicate with other devices even when they are not connected to the cloud.
4. **AWS IoT Analytics:** AWS IoT Analytics is a service that provides advanced analytics for IoT data. It allows you to clean, process, enrich, store, and analyze IoT data at scale.
5. **AWS IoT Events:** AWS IoT Events is a service that helps you detect and respond to events from IoT sensors and applications. It allows you to create custom event detection logic and take actions based on those events.
6. **AWS IoT Things Graph:** AWS IoT Things Graph is a service that makes it easy to visually connect different devices and web services to build IoT applications. It provides a drag-and-drop interface for creating IoT workflows.
7. **AWS IoT SiteWise:** AWS IoT SiteWise is a managed service that collects, structures, and searches IoT data from industrial equipment at scale. It allows you to monitor and visualize equipment data in real-time.
8. **AWS IoT Device Defender:** AWS IoT Device Defender is a service that helps you secure your IoT devices. It provides auditing, monitoring, and alerting for IoT device fleets to help you identify and respond to security issues.

These are just a few of the many AWS services that can be used to build IoT applications. AWS provides a wide range of tools and services to help developers create secure, scalable, and efficient IoT solutions.

NETCONF (Network Configuration Protocol) is a network management protocol used to manage network devices.

YANG (Yet Another Next Generation) is a data modeling language used to model configuration and state data manipulated by NETCONF.

Together, **NETCONF** and **YANG** provide a powerful framework for system management, allowing network administrators to perform various management tasks such as configuration, monitoring, and software updates in a standardized and efficient manner.

Here's how the process typically works:

1. **YANG Data Models:** YANG is used to define data models that describe the configuration and state data of network devices. These data models define the structure and hierarchy of the data that can be manipulated using NETCONF.
2. **NETCONF Protocol:** NETCONF is a network protocol that allows clients to remotely manage network devices. It provides operations for editing configuration data, retrieving operational data, and performing other management tasks.
3. **Configuration Changes:** Using NETCONF, administrators can make configuration changes to network devices by sending XML messages that conform to the YANG data models. These messages are sent over a secure connection to ensure confidentiality and integrity.
4. **Validation and Error Handling:** NETCONF provides mechanisms for validating configuration changes and handling errors. For example, if a configuration change violates a constraint defined in the YANG data model, the device will reject the change and provide an error message indicating the reason for the rejection.
5. **State Data Retrieval:** NETCONF also allows administrators to retrieve operational state data from network devices. This can be used for monitoring purposes or to gather information about the current state of the network.
6. **Software Updates:** NETCONF can be used to perform software updates on network devices. Administrators can use NETCONF to upload new software images to devices and instruct them to update their firmware.

Overall, NETCONF and YANG provide a standardized and efficient way to manage network devices, enabling administrators to perform a wide range of management tasks in a consistent and reliable manner.